📦 Project: Product List Manager

🎯 Project Goal

Create a **C# Console Application** that allows users to enter product names, validate them, store them in a list, and display the final result in a structured way.

The project is divided into **4 difficulty levels**.
Each level builds on the previous one.

🛠️ Technologies

- C# (.NET 8/9/10)
- Console Application
- Visual Studio / VS Code

📚 Learning Objectives

By completing this project, students will practice:

- Variables and data types
- Loops
- Conditional statements
- Lists and collections
- String handling
- Input validation
- Methods and modular programming
- Sorting and filtering
- Regular Expressions (Regex)
- Error handling
- Clean code principles

✅ **General Requirements**

- The application runs continuously until the user types:

exit

- The word exit should work regardless of:
    - Upper/lowercase letters
    - Leading/trailing spaces

Examples:

exit
EXIT
 Exit
  eXiT

🟢 Level 1 – Basic Product Input

Requirements

- Ask the user to enter product names.
- Store all entered products in a list.
- When the user types exit, display all products.

Concepts Practiced

- while loop
- List<string>
- Console input/output

🖥️ Sample Screenshot – Level 1

```
----------------------------------------
 PRODUCT LIST MANAGER - LEVEL 1
----------------------------------------

Enter product names.
Type 'exit' to finish.

Product: AB-1234
Product: XYZ-5000
Product: Milk
Product: Keyboard
Product: exit

Products entered:

- AB-1234
- XYZ-5000
- Milk
- Keyboard

Press any key to continue...
```

🟡 Level 2 – Sorting & Improved Exit Handling

Additional Requirements

- The application must accept:
    - exit
    - EXIT
    - ExIt
    - EXIT
- Trim unnecessary spaces.
- Sort the product list alphabetically before displaying it.

Concepts Practiced

- Trim()
- ToLower()
- Sorting lists
- String comparison

🖥️ Sample Screenshot – Level 2

```
----------------------------------------
 PRODUCT LIST MANAGER - LEVEL 2
----------------------------------------

Enter product names.
Type 'exit' to finish.

Product: PC-1234
Product: MAC-500
Product: LABAN
Product:    eXiT

Sorted product list:

- LABAN
- MAC-500
- PC-1234

Press any key to continue...
```

🟠 Level 3 – Product Validation

Additional Requirements

Now the application should only accept product names in this format:

LETTERS-NUMBER

Rules:

- Left side:
    - Letters only (A-Z)
- Right side:
    - Numbers only
    - Must be between 200 and 500

✅ Valid Examples

CE-400
XX-480
LABAN-231

❌ Invalid Examples

CE400
XX3-480
LABAN-100

Required Error Messages

Problem	Example Message
Empty input	`Input cannot be empty.`
Missing dash	`Product must contain a dash (-).`
Invalid left side	`The left side must contain letters only.`
Invalid number	`The right side must contain numbers only.`
Number out of range	`The numeric part must be between 200 and 500.`

Concepts Practiced

- Validation logic
- Methods
- int.TryParse()
- Regular Expressions
- Defensive programming

🖥️ Sample Screenshot – Level 3

```
----------------------------------------
 PRODUCT LIST MANAGER - LEVEL 3
----------------------------------------

Enter product names.
Type 'exit' to finish.

Product: AB-400
Product: AB-300
Product: -300
ERROR: The left side must contain letters only.

Product: AB-300B
ERROR: The right side must contain numbers only.

Product:
ERROR: Input cannot be empty.

Product: XY-350
Product: XY-700
ERROR: The numeric part must be between 200 and 500.

Product: Exit

Sorted valid products:

- AB-300
- AB-400
- XY-350

Press any key to continue...
```

🔴 Level 4 – Advanced Developer Challenge

🚀 Goal

Transform the application into a more realistic inventory management system.

Additional Requirements

1. Prevent Duplicate Products

If a product already exists:

WARNING: Product already exists.

---

2. Add Menu System

Example:

1. Add Product
2. View Products
3. Search Product
4. Delete Product
5. Exit

---

3. Search Feature

Users can search products by:

- Name
- Product number

Example:

Search: AB

Results:
- AB-300
- AB-400

---

4. Delete Feature

Example:

Enter product to delete: AB-300

Product removed successfully.

---

5. Statistics Section

Display:

- Total products
- Highest product number
- Lowest product number
- Average product number

---

6. Save to File (Bonus)

Save all products to:

products.txt

Or:

products.json

Concepts Practiced in Level 4

- Classes & Objects
- LINQ
- File handling
- Menus
- Searching algorithms
- CRUD operations
- Object collections

- Serialization
- Clean architecture

🖥️ Sample Screenshot – Level 4

```
========================================
 PRODUCT INVENTORY SYSTEM - LEVEL 4
========================================

1. Add Product
2. View Products
3. Search Product
4. Delete Product
5. Statistics
6. Exit

Select option: 1

Enter product: AB-300
Product added successfully.

Enter product: AB-300
WARNING: Product already exists.

----------------------------------------

Select option: 3

Search product: AB

Results:
- AB-300
- AB-450

----------------------------------------

Select option: 5

Statistics:
- Total Products: 5
- Lowest Number: 220
- Highest Number: 480
- Average Number: 341

----------------------------------------

Select option: 6

Saving products...
Application closed.
```

⭐ Optional Extra Challenges

Beginner+

- Color error messages red
- Color success messages green

Intermediate

- Use Regex for validation
- Add unit tests

🧠 Interview-Style Questions

Beginner Questions

1. What is the difference between List and Array?
2. What does int.TryParse() do?
3. Why should validation be separated into methods?

Intermediate Questions

1. What is encapsulation?
2. Why use classes instead of plain strings?
3. What is LINQ?

Advanced Questions

1. What design improvements would you make?
2. How would you store products in a database?
3. How would you unit test validation logic?

📌 Submission Requirements

Students should submit:

- Complete Visual Studio solution
- Source code

- README file
- Screenshot of running application

Optional:

- GitHub repository link

🏁 Final Goal

By the end of Level 4, students will have built a small but realistic inventory system similar to beginner-level industry projects.